

Vinith Ramaswamy

vramaswamy4.github.io | linkedin.com/in/vinithramaswamy | github.com/vramaswamy4 | justbookapp.com

vinithramaswamy@gmail.com | (608) 949-4944 | San Jose, CA | U.S. permanent resident (no sponsorship needed)

SUMMARY

Software engineer who has designed, shipped and operated a production platform alone: JustBook, a booking and payments SaaS live on web, iOS and Android, and Flagdown, a full-stack ride-sharing app on Go microservices and Next.js. Two UW–Madison B.S. degrees in three years, GPA 3.96. Looking for a backend, platform or full-stack role in the SF Bay Area.

EDUCATION

B.S. Computer Science & B.S. Data Science GPA 3.96 / 4.0

Aug 2023 – May 2026

University of Wisconsin–Madison · *Graduated with Distinction, in three years* · *Certificate in Entrepreneurship*

Coursework: Big Data Systems, Honors Algorithms, Databases, Machine Learning · full list: vramaswamy4.github.io/coursework

TECHNICAL SKILLS

Languages: Python, Go, SQL, JavaScript, TypeScript, Java, C++, C, R

Backend & data: Flask, SQLAlchemy, PostgreSQL, REST API design, gRPC, Redis, MongoDB, Kafka, RabbitMQ, Stripe

Infrastructure: Docker, Kubernetes, GitHub Actions, Prometheus, Grafana, OpenTelemetry, Render, AWS S3

Frontend & mobile: React 19, Next.js, Vite, Capacitor (iOS/Android), React Native, Figma

Testing: pytest, Vitest, Playwright, k6 load testing

EXPERIENCE

Founder & Software Engineer

May 2025 – Present

JustBook · *Booking and payments SaaS* · justbookapp.com · [App Store](#) · [Google Play](#) · [Write-up](#) · [Engine repo](#)

Stack: Python, Flask, SQLAlchemy, PostgreSQL, React 19, Capacitor, Stripe, Twilio, Render, AWS S3, pytest, Playwright

- **Took a booking and payments platform from idea to production alone in 15 months**, live on web, iOS and Android: a Flask/PostgreSQL API (633 endpoints, 127 models), a React 19 front end and Capacitor mobile apps.
- **Shipped the iOS and Android apps from the same React codebase** through Capacitor, with native Tap to Pay, push notifications, passkeys and barcode scanning, so one person maintains three clients.
- **Designed one availability engine for every booking surface** (public site, admin calendar, waitlist) so they can never disagree about a free slot; a constant 5 queries per month view, pinned by tests; published on GitHub.
- **Eliminated double bookings under concurrent load:** reproduced the race with a k6 load test, then serialized all five booking write paths with PostgreSQL advisory locks keyed per branch and day.
- **Cut the booking-flow read from nine sequential queries to one** (about 0.8 s cold before) with request-scoped memoization and a commit-invalidated cache; query-budget tests fail the build if the count grows.
- **Built the test infrastructure behind 8,500+ automated tests** (pytest on real PostgreSQL with an isolated database per run, Vitest, Playwright at desktop, phone and iOS-WebKit sizes with axe checks), so every release ships from green.
- **Replaced review-time policing with 22 custom build guards** that fail the build on secrets in logs, schema/migration drift, error-contract violations and stale documentation, so a standard cannot be broken silently.
- **Formed and run the company:** LLC, Stripe, Twilio and Meta approvals, Apple Developer organization, pricing, public docs and the marketing site; designed the first screens in Figma before building them.

IT Help Desk Technician, Level 1 to Lead

Jan 2024 – May 2026

UW–Madison Libraries · *Part-time* · *Madison, WI*

- **Promoted through four levels to Lead within a year**, one of eight leads running the public labs and help desk for 50+ student technicians handling 100+ tickets a day, 29 hours a week alongside a full-time dual degree.
- **Supported library staff across campus IT systems:** Active Directory group membership and permissions, InfoBlox DNS and IP management, Bomgar remote sessions, printers and print servers, and a Redmine ticket queue.

PROJECTS

Flagdown Full-stack ride-sharing app in Go microservices and Next.js

2026

github.com/vramaswamy4/flagdown · [Write-up](#)

Stack: Go, gRPC, RabbitMQ, Kafka, Redis, MongoDB, Kubernetes, Prometheus, Grafana, OpenTelemetry, Next.js

- **Built five Go microservices and a Next.js front end on Kubernetes**, with gRPC where the caller needs an answer, RabbitMQ where it only needs to announce an event, and WebSockets for live rider and driver updates.
- **Made the driver service stateless** by streaming positions through Kafka (keyed by driver ID) into an idempotent consumer that keeps a Redis geo index, so nearest-driver matching is one GEOSEARCH on any replica.
- **Proved crash-safety instead of claiming it:** durable queues, acks after the work, backoff retries, dead-lettering and idempotent handlers, verified by a chaos test that kills a pod mid-ride and asserts no double-processing.